LLM 기반 학사 정보 Agent 시스템 구현 보고서

서론 - LLM 애플리케이션 개발의 한계

초기 LLM 기반 애플리케이션은 단순한 API 호출 구조에 의존한다. 즉, 입력 → LLM → 출력의 단일 흐름으로 구성된다. 그러나 이러한 구조는 외부 정보 접근 불가 (최신 정보 반영 어려움), 복잡한 문제 해결 불가능 (단계적 추론 부족), 도구 사용 불가능 (계산, 검색, 검증 불가), 응답 구조가 고정됨 (유연성 부족) 이라는 한계를 가진다 따라서 단순 API 호출 방식은 실제 서비스 수준의 문제 해결에 한계가 있다.

오케스트레이션 프레임워크의 필요성

이러한 문제를 해결하기 위해 등장한 것이 LangChain, LangGraph 와 같은 오케스트레이션 프레임워크이다. 이 프레임워크들은 Chain 기반 파이프라인 구성, RAG 기반 외부 지식 결합, Tool 기반 행동 확장, Agent 기반 자율 의사결정과 같은 기능을 제공한다. 즉, LLM 을 단순 생성기가 아닌 "문제 해결 엔진"으로 확장한다.

도메인 적합성

프로젝트의 도메인은 "대학 학사 정보 시스템"이다. 이 도메인은 다음 이유로 Chain/RAG/Agent 모두 필요하다

- 학칙 → RAG 필요 (문서 검색)
- 학사 일정 → Tool 필요 (정형 데이터)
- 졸업 요건 → 계산 필요 (논리/조건 판단)
- 과목 추천 → Agent 필요 (다중 추론)

따라서 단일 방식으로 해결 불가능한 복합 도메인이다.

배경 지식

Prompt Engineering (Few-shot 포함)

Few-shot prompting 은 예시를 통해 모델의 출력 형식을 유도하는 방식이다.

project > compare > benchmark.py > ...

```python
from rag.rag_chain import rag_chain
from agent.react_agent import agent

questions = [

    "졸업 가능?",
    "개강 언제?",
    "전필 부족하면?",
    "교양 부족인데 추천해줘",
    "학칙 기준 졸업요건"

]

for q in questions:

    print("=" * 60)
    print("QUESTION:", q)

    print("\n[RAG Chain]")

    rag_result = rag_chain.invoke(q)

    print(rag_result)

    print("\n[ReAct Agent]")

    agent_result = agent.invoke({

        "messages":[
            ("user", q)
        ]

    })

    print(
        agent_result["messages"][-1].content
    )

    print()
```

효과로는 출력 스타일 안정화, Tool 호출 유도가 강화된다는 점이 있다.

LCEL (LangChain Expression Language)

LCEL 은 다음과 같은 선언적 합성 구조를 가진다 prompt → model → parser

Unix Pipe 와 유사성

cat data | grep "keyword" | sort , 동일하게 LCEL 은 데이터 흐름을 파이프처럼 연결한다.

Chain / RAG / Agent 차이

구조	특징	사용 상황
Chain	고정 흐름	단순 변환
RAG	검색 + 생성	문서 기반 질문
Agent	자율 판단	복합 문제 해결

ReAct 패턴

ReAct 는 Thought → Action → Observation → Answer 구조를 따르며 LLM 이 생각하고 도구를 사용하고, 결과를 관찰한 후 답변을 생성하는 것이 핵심이다.

@tool 변환 구조

Python 함수 → LLM 이 이해 가능한 Tool Schema 로 변환

docstring = LLM 의 선택 기준

시스템 설계

전체 아키텍처 구조

User
↓
ReAct Agent (LangGraph)
↓
LLM Decision Layer
↓

Tool Selection

├─── RAG (FAISS)

├─── Academic Schedule Tool

├─── Graduation Calculator

└─── Rule Checker

↓

Final Answer

Custom Tools 설계

Tool	역할
academic_schedule	학사 일정 조회
calculate_graduation	졸업 학점 계산
rag_search	학칙 검색

RAG 구성

Loader: WebBaseLoader

Splitter: RecursiveCharacterTextSplitter

Vector DB: FAISS

Embedding: text-embedding-3-small

chunk 설정 근거

chunk_size=500, overlap=100

학칙 문단 구조를 유지하며 의미 손실을 방지한다.

Agent Tool 선택 흐름

User Query

→ Intent Classification

→ Tool Selection

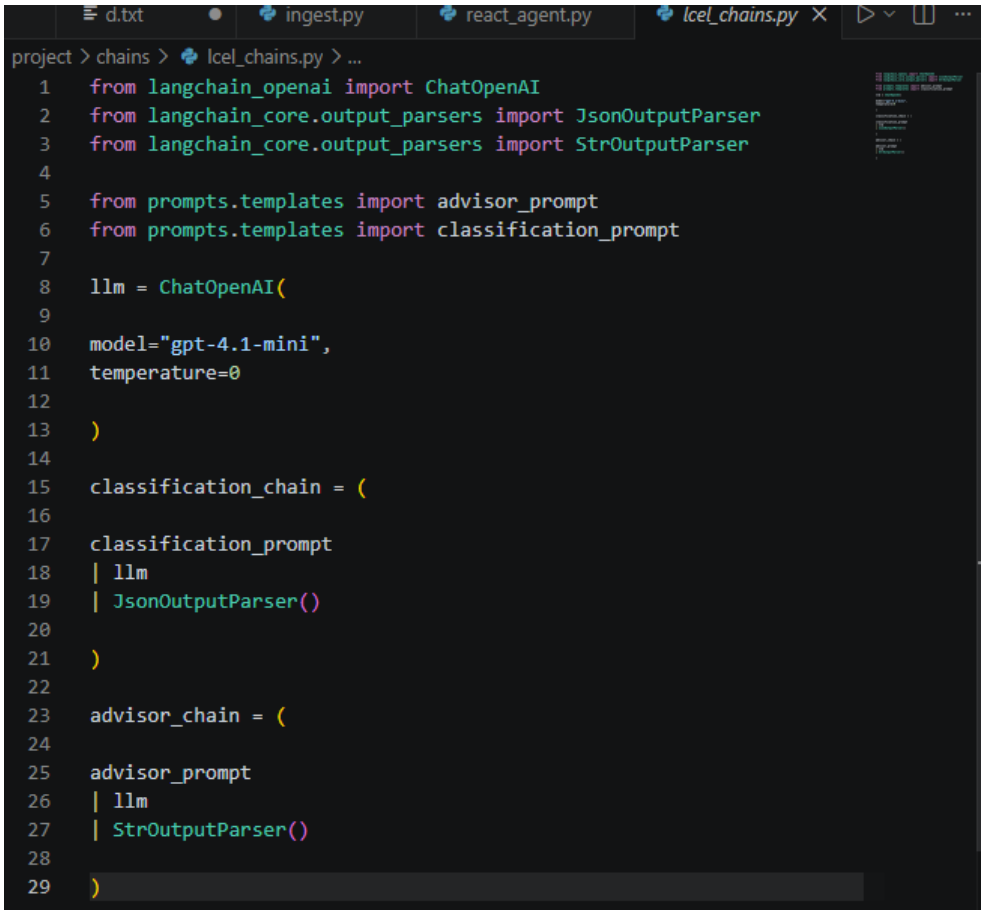
→ Tool Execution

→ Observation

→ Final Answer

구현 상세

LCEL + OutputParser



구조화된 출력을 가능하게 함

Sequential Chain

sequential.py

project > chains > sequential.py > ...

```python
from langchain_core.runnables import RunnableLambda

def analyze_credit(x):

    return {

    "missing":max(
        0,
        132 - x["total"]
    ),

    "data":x
    }

def recommend(x):

    if x["missing"]==0:

        return "졸업 가능"

    return f"{x['missing']}학점 부족"

seq_chain = (

RunnableLambda(analyze_credit)
|
RunnableLambda(recommend)

)
```

step1 = llm.invoke("핵심 키워드 추출")
step2 = llm.invoke("요약 생성")
final = llm.invoke(step1 + step2)

단계 분리를 통한 품질 향상

RAG vs Raw 구현 비교

항목	Raw Python	LangChain
코드량	많음	적음
유지보수	어려움	쉬움
디버깅	불편	trace 가능

Tool docstring 실험 결과

모호한 docstring 으로 Tool 선택 실패 증가와 RAG 과다 호출되는 현상이 발생하는 반면 명확한 docstring 은 정확한 tool 선택과 불필요 호출이 감소된다. 결론적으로 docstring 은 LLM 의 API 문서 역할을 한다.

ReAct Agent 구성

```
agent = create_react_agent(

llm,

tools=[

calculate_graduation,
academic_schedule,
handbook_search

]

)
```

결과 및 분석

Chain vs Agent 비교 (5 cases)

질문	Chain	Agent	Tools	결과
개강 언제	빠름	동일	schedule	동일
졸업 가능	단순	계산+판단	calc+rag	Agent 우수
학칙 질문	검색	구조 분석	RAG	Agent 우수
학점 부족	설명	계산	calc	Agent 우수
과목 추천	제한	전략적	multi-tool	Agent 우수

Agent 성공 사례로는 multi-tool chaining, RAG + 계산 결합, 구조화 출력 성공이 있으며 실패 사례는 불필요한 tool 호출, 단순 질문에도 RAG 사용, tool loop 발생하는 점이 있다. 확장 기능 효과는 3 가지이다. Streaming → UX 개선, LangSmith → 디버깅 가능, Structured output → 후처리 가능의 효과가 있다.

고찰

생각하고 행동하는 AI 만들기 과제의 프로젝트를 구현하면서 Chain, RAG, Agent 는 경쟁 관계가 아니라 문제 유형에 따라 선택해야 하는 서로 다른 추상화 계층이라는 점을 확인할 수 있었다. 단순 정보 변환이나 반복적인 입력 처리에서는 고정된 흐름을 가지는 Chain 방식이 구현과 디버깅 측면에서 효율적이었다. 반면 학칙 문서 검색과 같이 외부 지식이 필요한 상황에서는 RAG 구조가 효과적이었으며, 문서 검색 결과를 기반으로 응답을 생성함으로써 환각 현상을 줄일 수 있었다. 그러나 사용자의 질문이 단순 검색을 넘어 계산, 검색, 판단을 동시에 요구하는 경우에는 Agent 구조가 더 적합했다. 예를 들어 졸업 가능 여부 판단과 같이 학점 계산, 규정 검색, 결과 해석이 함께 필요한 문제에서는 Agent 가 상황에 맞는 도구를 선택하고 순차적으로 활용할 수 있다는 장점을 보였다.

다만 Agent 기반 시스템은 여러 한계도 드러냈다. 먼저 Tool 호출과 추론 단계를 반복하기 때문에 Chain 기반 시스템보다 비용이 증가하고 응답 시간이 길어졌다. 또한 동일한 입력에서도 도구 선택 경로가 달라질 수 있는 비결정성이 존재하여 테스트와 디버깅 난이도가 높았다. 일부 사례에서는 단순 질문에도 불필요한 Tool 을 호출하거나, 검색 도구를 과도하게 사용하는 현상도 관찰되었다. 따라서 실제 서비스 환경에서는 Tool 호출 횟수 제한, 비용 모니터링, 실패 시 fallback 응답 제공과 같은 운영 전략이 필수적이라고 판단하였다.

향후 개선 방향으로는 단일 Agent 구조를 넘어 역할을 분리한 Multi-Agent 협업 구조를 고려할 수 있다. 예를 들어 검색 담당 Agent 와 분석 담당 Agent 를 분리하면 책임이 명확해지고 성능 향상을 기대할 수 있다. 또한 Self Reflection 또는 Self Critique 기반 자기 검증 단계를 추가하여 잘못된 Tool 호출이나 부정확한 응답을 줄일 수 있을 것으로 보인다. 마지막으로 Dynamic Tool Discovery 와 Memory 기반 개인화를 도입한다면 사용자의 이전 대화와 상황을 반영하는 보다 적응적인 학사 상담 시스템으로 확장할 수 있을 것이다.